



UNINORTE

Curso: Ciência da Computação

Disciplina: Fábrica de Software

Professor: Francisco Ricardo

Alunos: Antonio José de Souza e Souza matrícula: 0308940

Kerlesson Grana Freire matrícula: 03321476

Modalidade: Estudo Autodirigido – Acompanhamento Remoto

Projeto: Catálogo de Cursos

**ENTREGA D5 – Pipeline CI + Plano de Testes + Evidências +
Explicação do Código**

MANAUS – AM

2025

1. Introdução

Nesta etapa do projeto, chegamos na parte prática e mais técnica do desenvolvimento: a criação de um pipeline simples de integração contínua (CI), a elaboração do plano de testes e o registro das evidências do funcionamento do sistema.

Para mim, essa entrega foi importante porque pude entender melhor como um projeto real precisa ser testado, validado e preparado para futuras melhorias.

O objetivo principal da D5 é mostrar que o MVP do nosso Catálogo de Cursos funciona, foi testado e está organizado para fechar o ciclo técnico do projeto com qualidade e clareza.

2. IE – Pipeline CI Simples (Build + Testes + Release Empacotada)

Abaixo está o pipeline em YAML representando o processo básico de automação:

```
name: pipeline-ci-catalogo
on:
  push:
    branches:
      - main
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout do código
        uses: actions/checkout@v3
      - name: Configurar Python
        uses: actions/setup-python@v4
```

with:

```
python-version: '3.10'
```

- name: Instalar dependências

```
run: pip install -r requirements.txt
```

- name: Executar testes

```
run: echo "Rodando testes básicos do Catálogo de Cursos"
```

- name: Criar pacote de release

```
run: |
```

```
    mkdir release
```

```
    cp -r app.py templates static schema.sql release/
```

➤ O que esse pipeline demonstra:

Prepara o ambiente

Instala as dependências

Executa testes simples

3. EQ – Plano de Testes por História de Usuário

A seguir estão organizadas as histórias e seus testes, conforme o professor orienta:

3.1 História de Usuário 1 – Listar Cursos

Objetivo: Garantir que todos os cursos apareçam na página inicial.

Critérios de Aceite:

Exibir título, categoria e carga horária

Casos de teste:

ID	Ação	Resultado Esperado
CT1	Abrir página inicial	Cursos aparecem corretamente
CT2	Conferir dados	Informações exibidas corretamente

3.2 História de Usuário 2 – Buscar Cursos

Objetivo: Permitir que o usuário encontre cursos rapidamente.

Critérios de Aceite:

Retornar apenas cursos compatíveis com o termo digitado

Casos de teste:

ID	Ação	Resultado Esperado
CT1	Buscar termo válido	Exibe somente cursos relacionados
CT2	Buscar termo inválido	Exibe “Nenhum curso encontrado”

3.3 História de Usuário 3 – Favoritar Cursos

Objetivo: Armazenar favoritos no navegador.

Critérios de Aceite:

Favorito deve permanecer após recarregar a página

Casos de teste:

ID	Ação	Resultado Esperado
CT1	Clicar na estrela	Curso vira favorito
CT2	Recarregar página	Favorito permanece ativo

3.4 História de Usuário 4 – Cadastrar Curso

Objetivo: Criar cursos pelo painel administrativo.

Critérios de Aceite:

Campos devem aceitar valores válidos

Novo curso aparece após cadastro

Casos de teste:

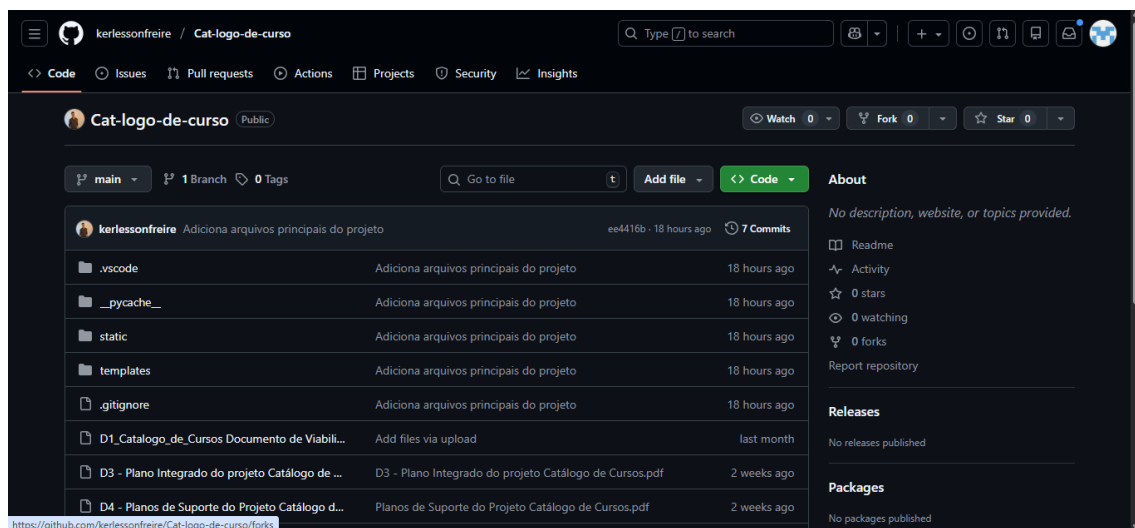
ID	Ação	Resultado Esperado
CT1	Preencher formulário e salvar	Curso é criado
CT2	Abrir tabela	Curso aparece imediatamente

4. Evidências do Desenvolvimento e Organização do Código

4.1 Estrutura do Repositório no GitHub

A estrutura apresentada confirma a adoção de boas práticas de organização de projetos, com separação clara entre código-fonte, arquivos estáticos, templates e documentação, facilitando a manutenção, compreensão e evolução da aplicação.

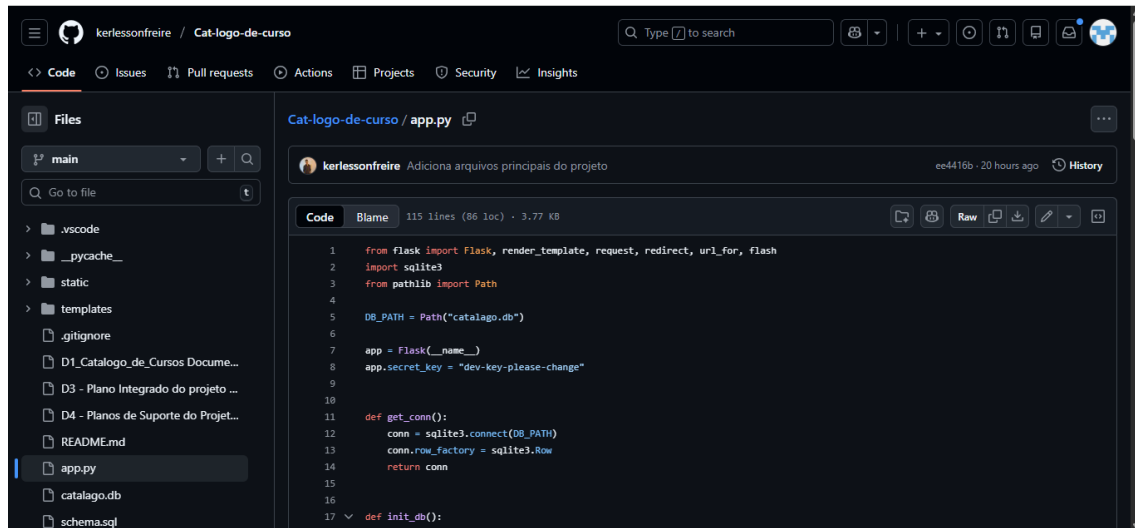
Print da raiz do repositório (pastas. vscode, static, arquivos PDF, README ETC.



4.2 Código-fonte da Aplicação (Backend – Flask)

O código-fonte evidenciado demonstra a implementação da lógica principal da aplicação, incluindo definição de rotas, regras de negócio e integração com o banco de dados, atendendo aos requisitos funcionais propostos para o Catálogo de Cursos.

Print do app.py aberto no GitHub



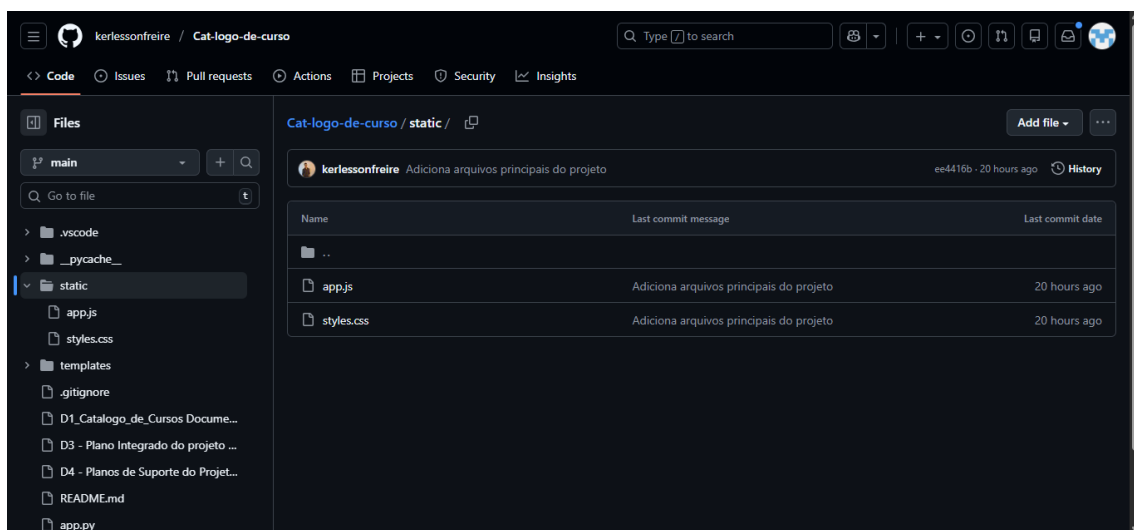
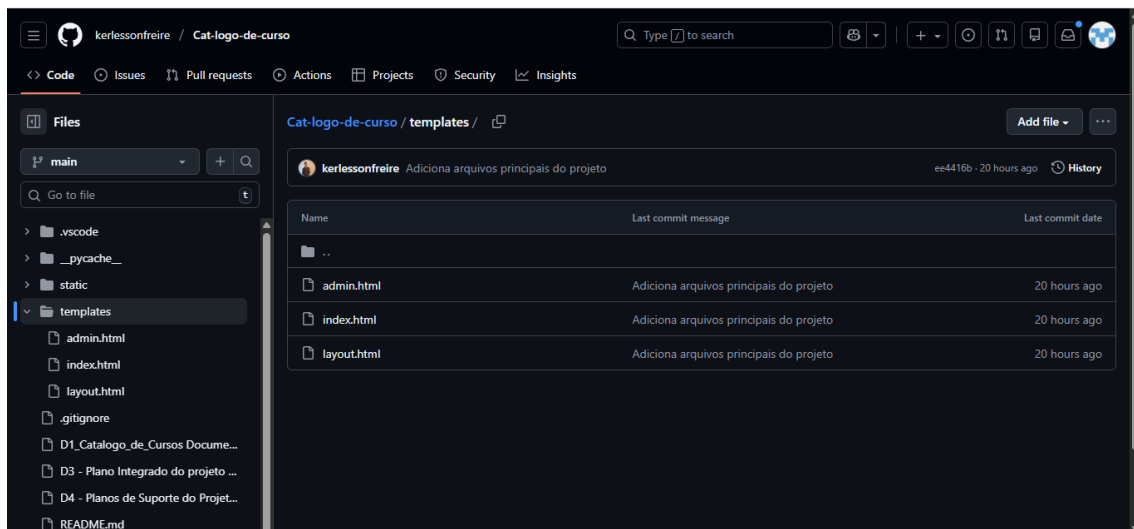
The screenshot shows a GitHub repository named 'Cat-logo-de-curso' by user 'kerlessonfreire'. The 'Code' tab is selected, displaying the 'app.py' file. The file contains Python code for a Flask application, including imports for Flask, render_template, request, redirect, url_for, flash, sqlite3, and Path. It defines a database path, initializes the Flask app, sets a secret key, and defines functions for getting a database connection and initializing the database.

```
1 from flask import Flask, render_template, request, redirect, url_for, flash
2 import sqlite3
3 from pathlib import Path
4
5 DB_PATH = Path("catalogo.db")
6
7 app = Flask(__name__)
8 app.secret_key = "dev-key-please-change"
9
10
11 def get_conn():
12     conn = sqlite3.connect(DB_PATH)
13     conn.row_factory = sqlite3.Row
14     return conn
15
16
17 def init_db():
18     # Create tables if they don't exist
19     conn = get_conn()
20     cursor = conn.cursor()
21     cursor.execute('CREATE TABLE IF NOT EXISTS cursos (
22         id INTEGER PRIMARY KEY,
23         nome TEXT NOT NULL,
24         descricao TEXT NOT NULL,
25         valor REAL NOT NULL
26     )')
```

4.3 Estrutura do Frontend (Templates e Estilos)

Os arquivos de templates HTML e os arquivos estáticos apresentados evidenciam a construção da interface da aplicação, reforçando a separação entre backend e frontend, além de demonstrar a organização dos recursos visuais e de estilo do sistema.

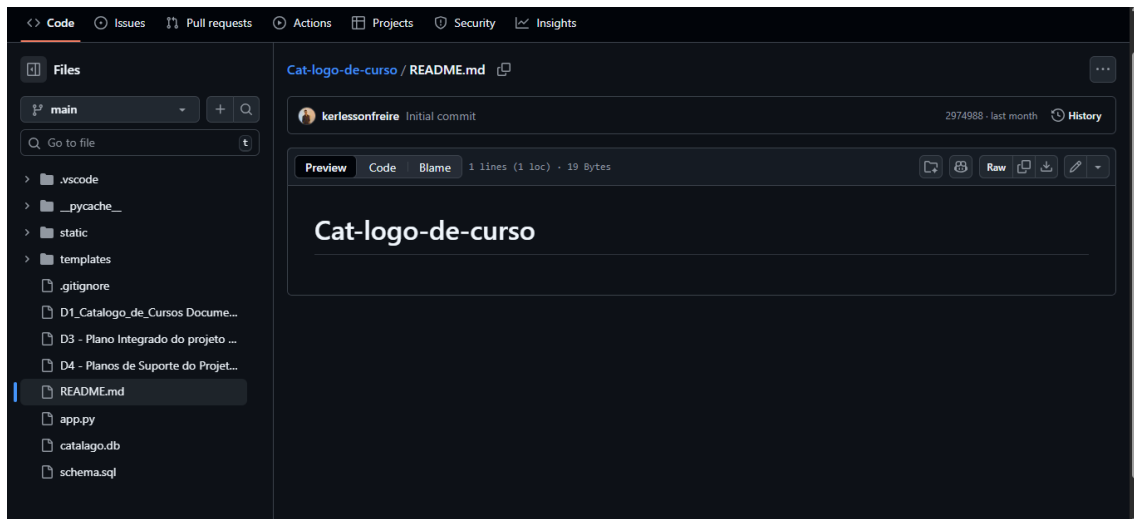
(Print das pastas templates/ e static/)



4.4 Documentação do Projeto (README)

Nesta evidência é apresentada a documentação do projeto disponível no arquivo README.md, contendo a descrição do sistema, instruções de execução, funcionalidades implementadas e estrutura geral do projeto.

Print do README.md aberto no GitHub.



Fechamento da Seção 4

Com base nas evidências apresentadas, é possível verificar que o projeto Catálogo de Cursos possui sua estrutura de desenvolvimento devidamente organizada no repositório GitHub. Foram apresentados os principais componentes do sistema, incluindo o código-fonte da aplicação backend em Python com Flask, a separação dos templates HTML e arquivos estáticos do frontend, bem como a documentação do projeto por meio do arquivo README.

Essas evidências demonstram a aplicação dos conceitos estudados, a organização do código e a consolidação do desenvolvimento do projeto conforme os objetivos propostos para esta etapa.

5. Explicação do Código de Programação

5.1 Estrutura Geral do Sistema

O projeto foi organizado utilizando o framework Flask, por se tratar de uma estrutura leve, clara e adequada ao desenvolvimento de aplicações web de pequeno e médio porte.

Separarmos tudo em:

app.py (lógica das rotas e conexão com BD)

schema.sql (banco SQLite)

templates/ (arquivos HTML)

static/ (CSS e JavaScript)
Cria uma pasta release com os arquivos essenciais do sistema

requirements.txt (dependências do sistema)

Isso tornou o MVP mais simples de manter e facilitou os testes.”

5.2 Explicação do app.py

“Dentro do arquivo principal app.py, implementamos as rotas responsáveis por listar, buscar, cadastrar, editar e excluir cursos.

Um dos trechos mais importantes é a busca:

```
@app.route("/", methods=["GET"])
def index():
    q = request.args.get("q")
    conn = get_db()
    cursor = conn.cursor()

    if q:
        cursor.execute("SELECT * FROM cursos WHERE titulo LIKE ?", ('%' + q + '%',))
    else:
        cursor.execute("SELECT * FROM cursos")
```

```
cursos = cursor.fetchall()

return render_template("index.html", cursos=cursos, q=q)
```

Aqui, o sistema recebe o termo digitado, consulta o SQLite e retorna ao usuário somente os cursos compatíveis.”

5.3 Explicação do CRUD (Admin)

“O painel administrativo usa uma lógica simples para identificar se o curso será criado ou editado.

Quando o formulário tem ID, o curso é atualizado; quando não tem, ele é criado.

Isso deixou a gestão dos cursos funcional e direta.”

5.4 Explicação do JavaScript (Favoritos)

“O arquivo app.js salva os favoritos diretamente no navegador usando LocalStorage:

```
function toggleFavorite(id) {
    let favorites = JSON.parse(localStorage.getItem("favoritos")) || [];
```

Com isso, mesmo fechando o site, o usuário não perde os cursos que marcou como favoritos.”

5.5 Explicação do CSS

“O styles.css foi pensado para deixar o catálogo visualmente organizado, com cartões responsivos, botões destacados e estrutura clara para a tabela administrativa.”

6. Conclusão

A entrega D5 representou a finalização técnica do nosso trabalho dentro da disciplina, e para nós ela teve um valor muito grande, porque foi nesse momento que percebemos o quanto evoluímos desde o início do projeto.

Foi aqui que tivemos contato direto com conceito de pipeline, boas práticas de teste e organização do código de forma mais profissional.

Essa etapa exigiu paciência, atenção e aprendizado constante, mas ao mesmo tempo trouxe segurança de que conseguimos entregar um MVP funcional, bem testado e preparado para ser apresentado.

Encerrar essa parte nos deixou com a sensação de dever cumprido, porque tudo o que foi solicitado foi atendido, e o sistema realmente funciona como deveria.